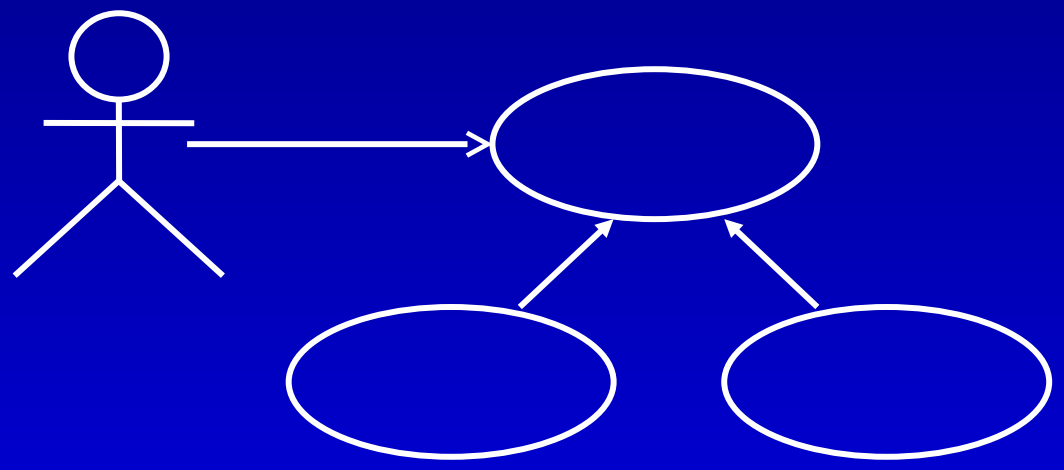
Use-Case-Driven Development



Use-Case Model

Copyright © 1998 Rational Software, all rights reserved

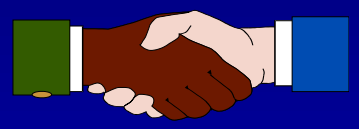
R

Objectives: Use-Case-Driven Development

- Explain the relationship between actors and system scope
- Describe the use-case model
- Define requirement attributes and explain their uses
- Explain how use cases drive analysis and design, test, and iteration planning

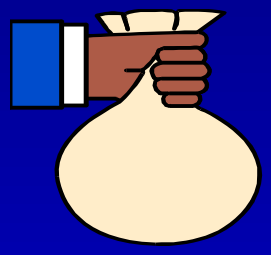
Copyright © 1998 Rational Software, all rights reserved

Where Do Requirements Come From?

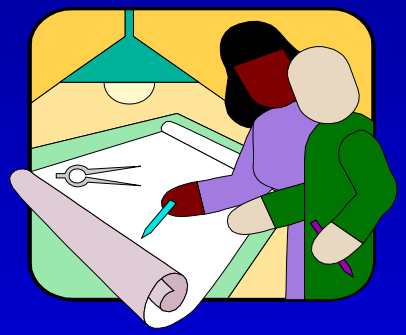


Partners

Requirement Specs
Business Plans
Personal Goals



Customer

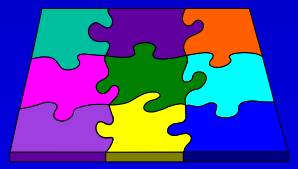


Analyst

Users

Problem Reports
Change Requests

Domain Experts
Industry Analysts
Site Visits
Business Model

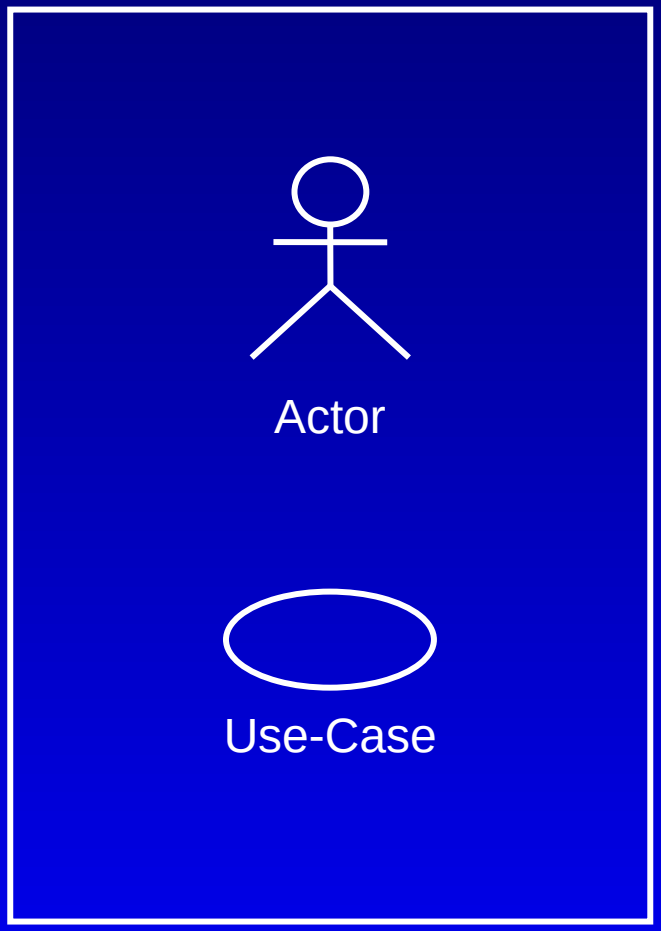


Problem Domain

Copyright © 1998 Rational Software, all rights reserved

R

Review of Major Concepts: Actors and Use Cases

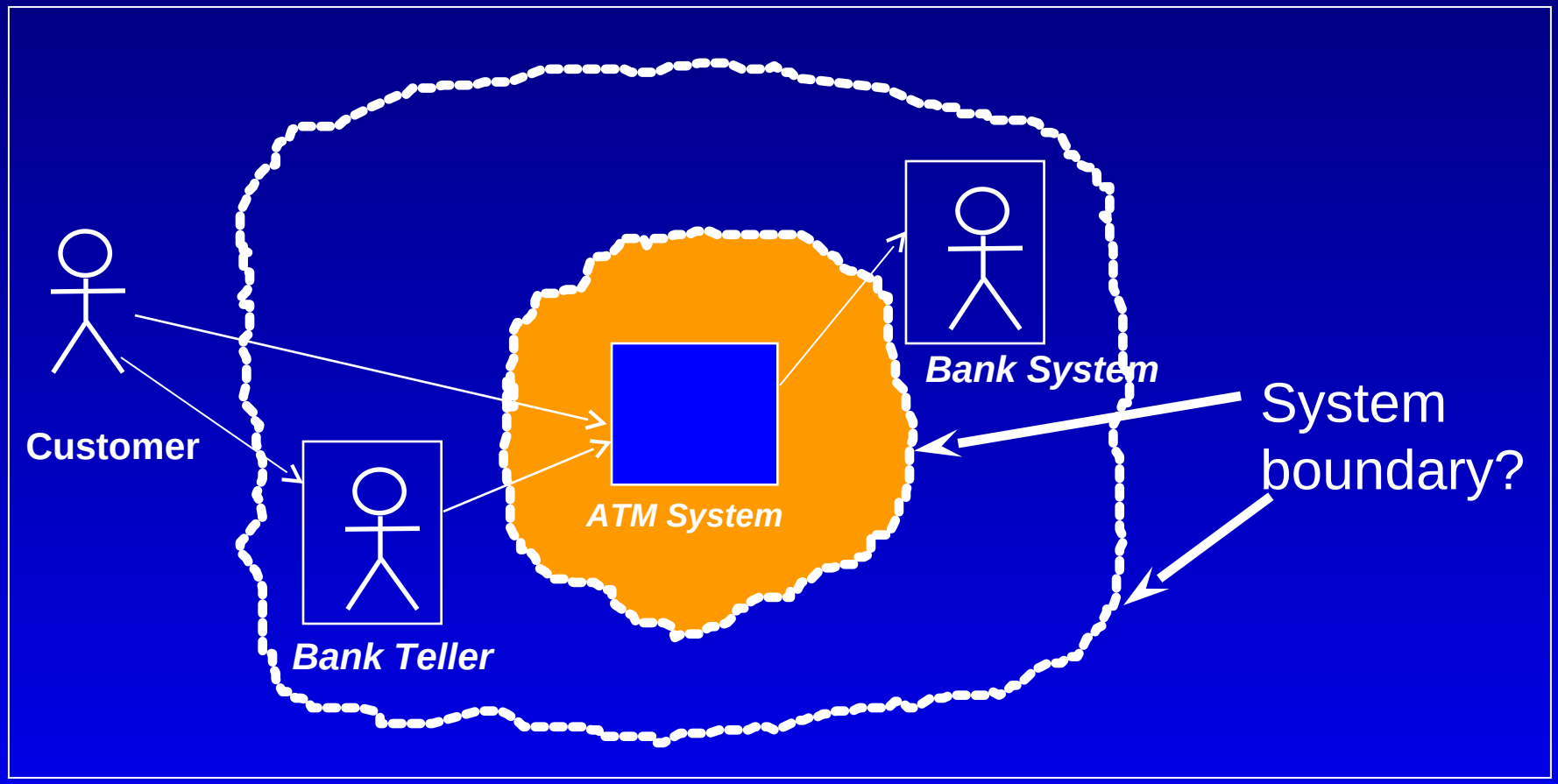


An **actor** is someone or something outside the system that interacts with the system

A **use case** is a sequence of actions a system performs that yields an observable result of value to a particular actor

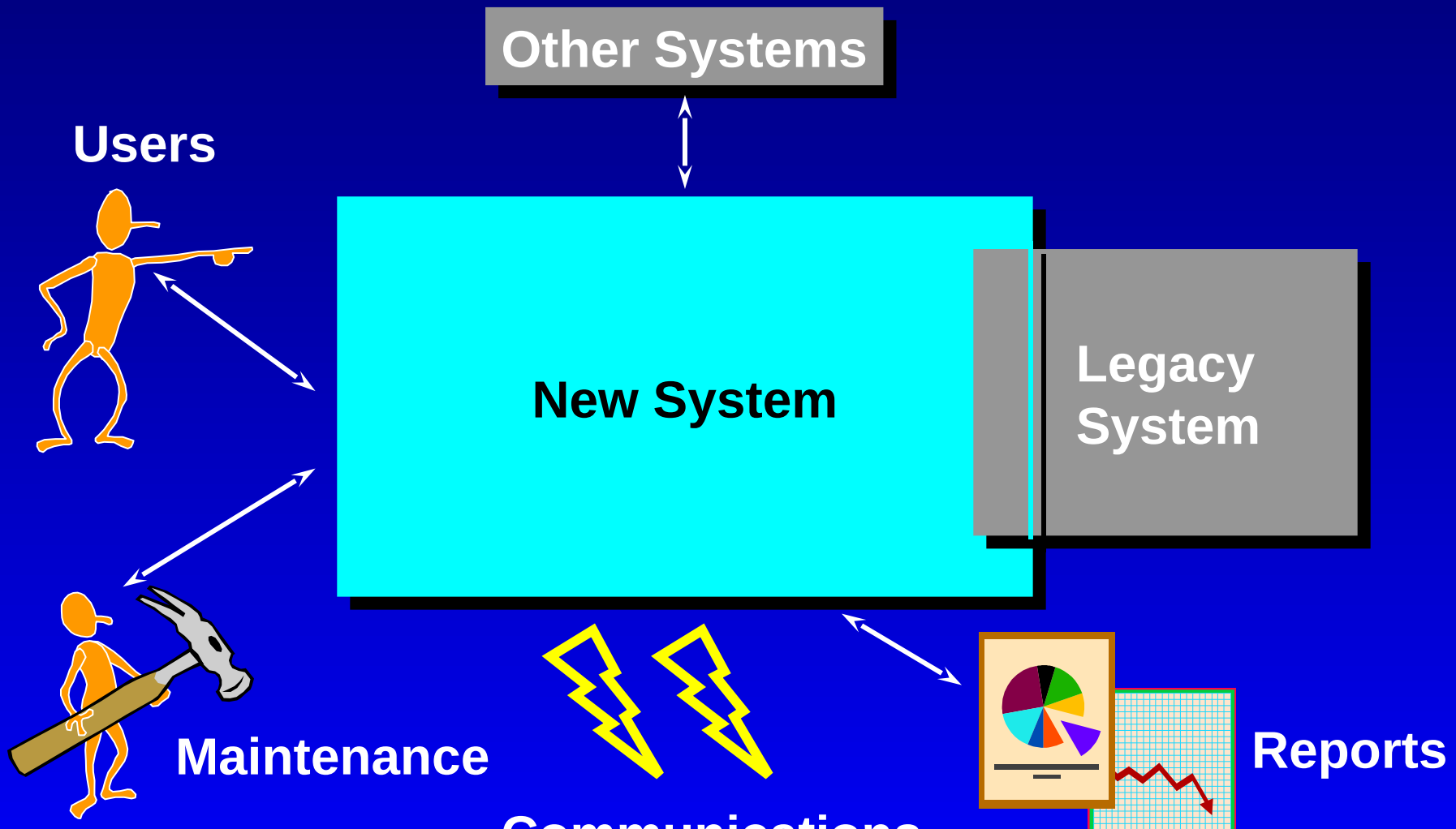
Copyright © 1998 Rational Software, all rights reserved

Actors and System Scope



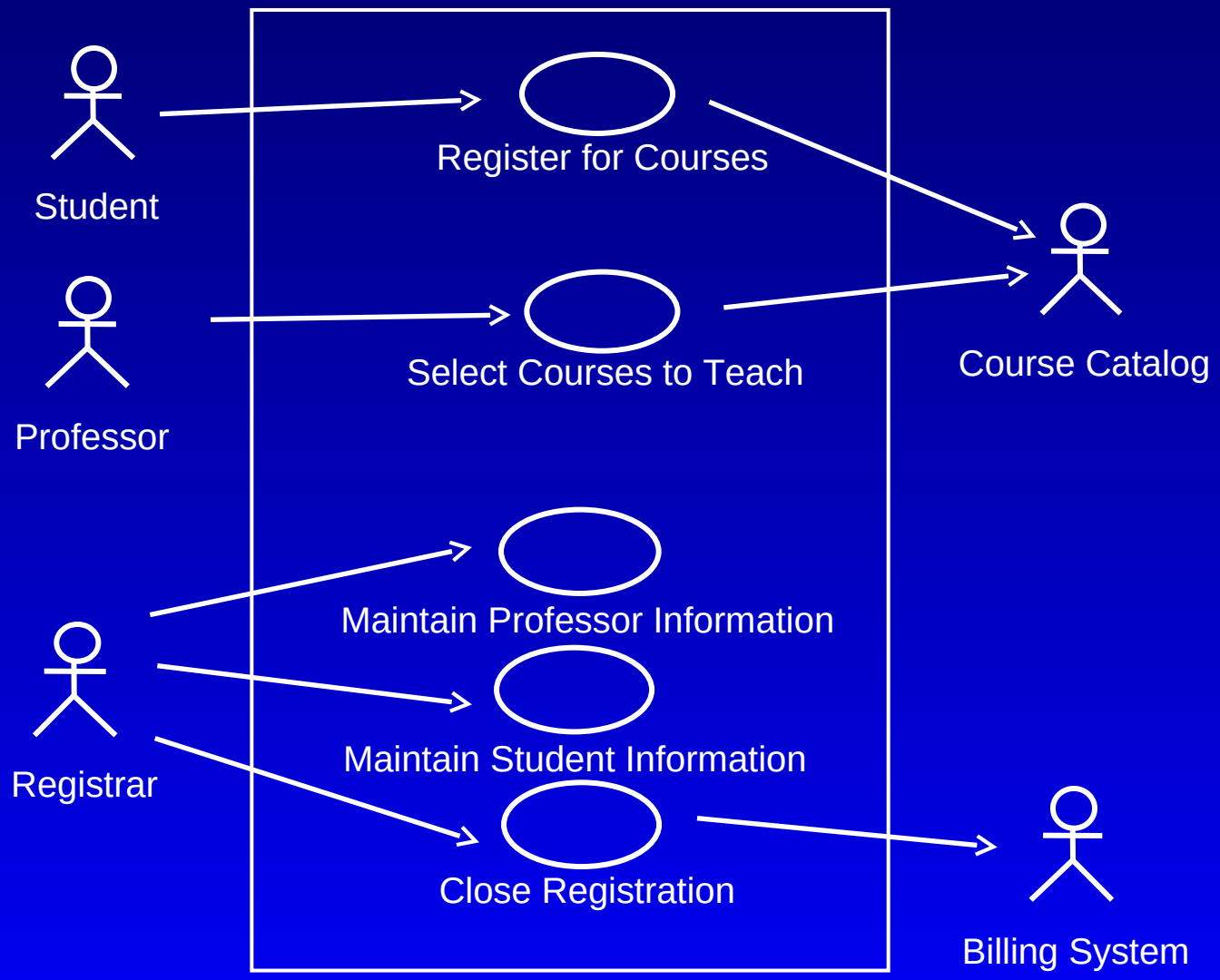
Copyright © 1998 Rational Software, all rights reserved

Defining System Boundaries



Copyright © 1998 Rational Software, all rights reserved

Example: Use-Case Diagram



Copyright © 1998 Rational Software, all rights reserved

Sample Flow of Events

- Flow of events for the Register for Courses Use Case
  1. The Student selects “create schedule.”
  2. The system displays a blank schedule form.
  3. The system retrieves a list of available course offerings from the Course Catalog System.
  4. The student selects 4 primary course offerings and 2 alternate course offerings from the list of available offerings. Once the selections are complete the student selects “submit.”
  5. The “Add Course Offering” sub-flow is performed at this step for each selected course offering.
  6. The system saves the schedule.

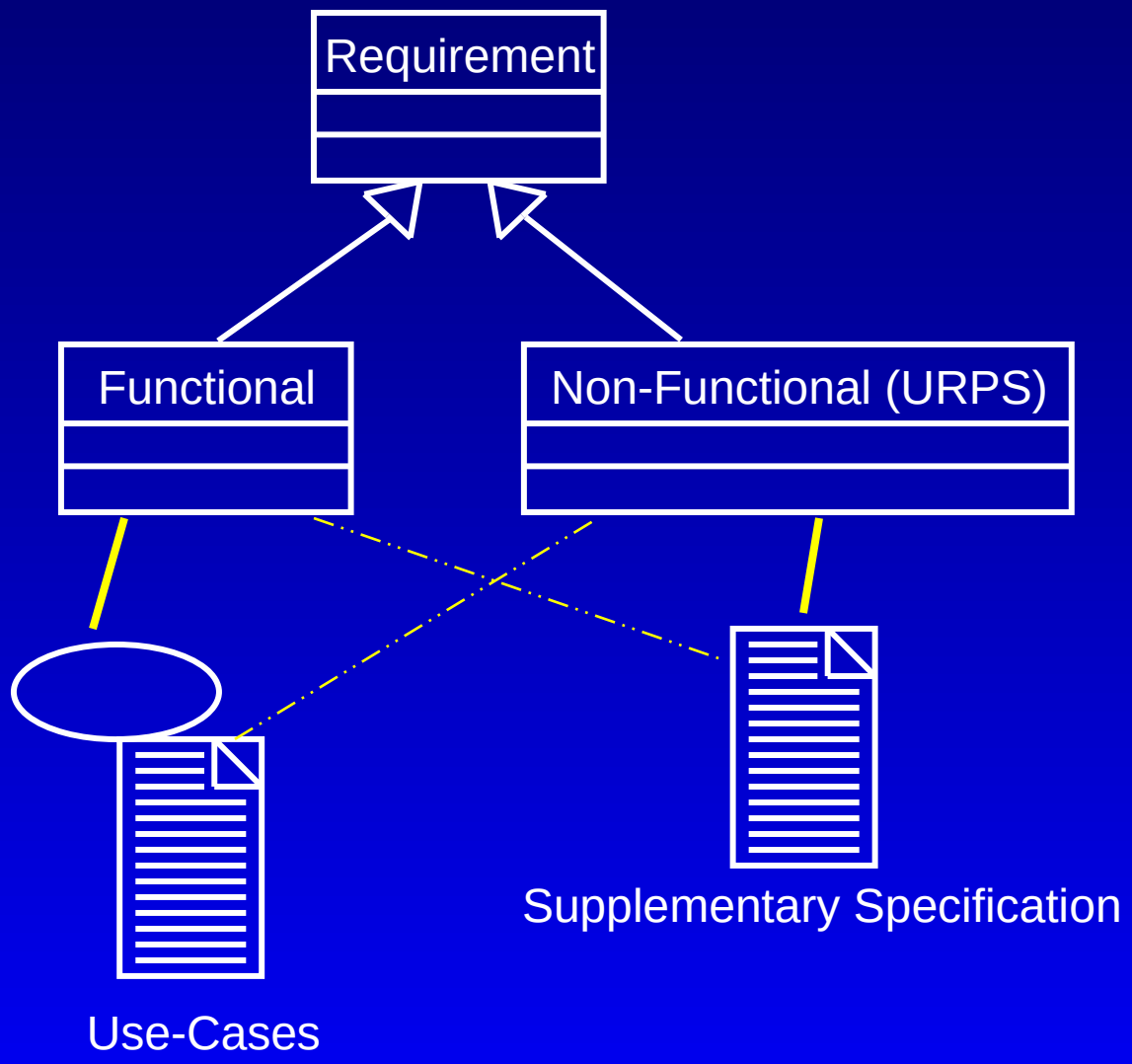
Copyright © 1998 Rational Software, all rights reserved

Types of Requirements

Functionality	Feature Set Capabilities	Generality Security
Usability	Human Factors Aesthetics	Consistency Documentation
Reliability	Frequency/Severity of Failure Recoverability	Predictability Accuracy MTBF
Performance	Speed Efficiency Resource Usage	Throughput Response Time
Supportability	Testability Extensibility Adaptability	Maintainability Compatibility Configurability
Design Constraints	Platform Language . . .	

Copyright © 1998 Rational Software, all rights reserved

Primary Requirements Artifacts



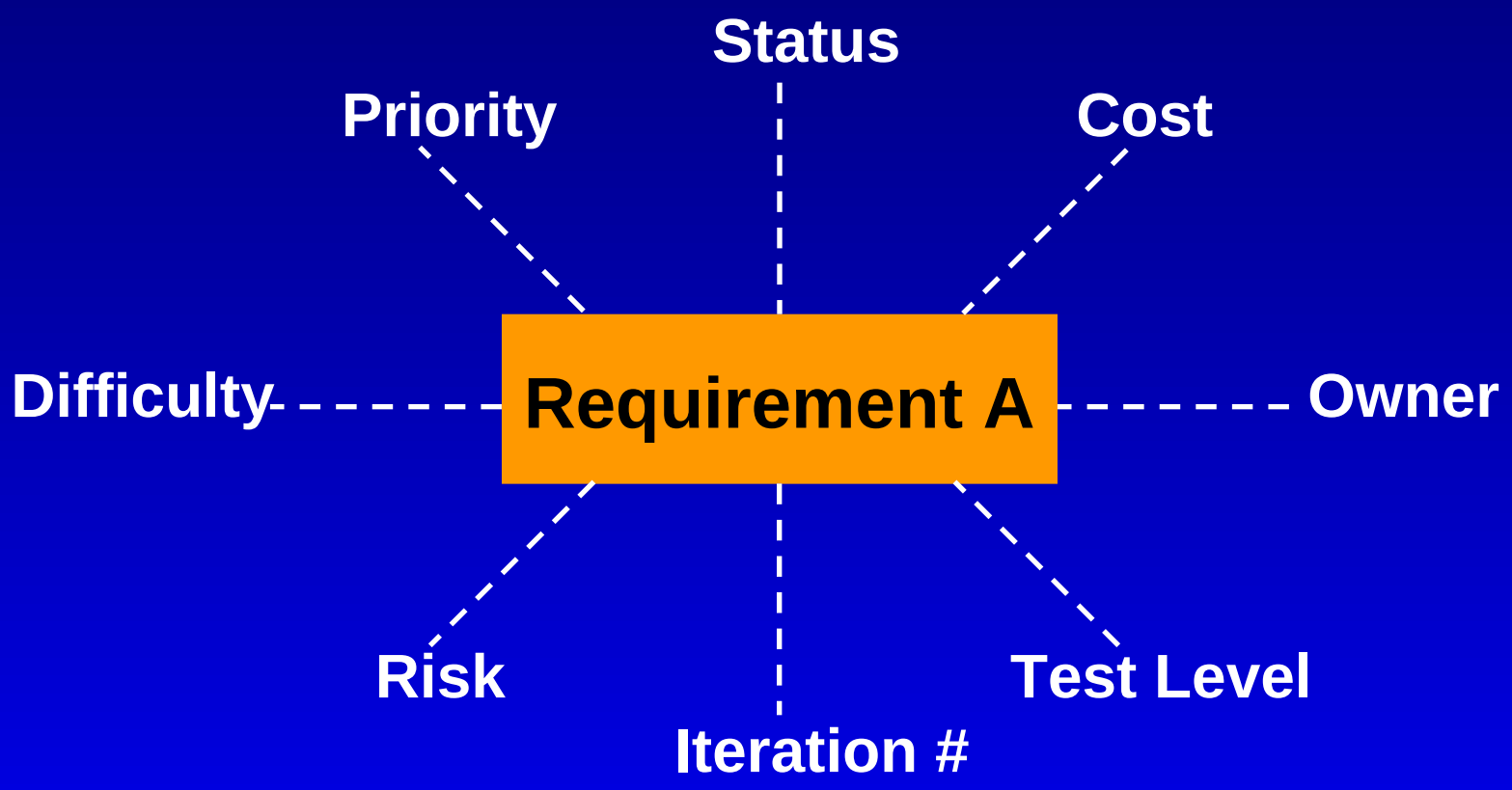
Copyright © 1998 Rational Software, all rights reserved

What Is in a Use-Case Model?

- Actors and their descriptions
- Use-Case diagrams showing relationships
- For each use case
  - Name and brief description
  - Flow of events
  - Preconditions and postconditions (optional)
  - Special requirements
  - Other diagrams, such as activity or state diagrams

Copyright © 1998 Rational Software, all rights reserved

Using Requirement Attributes



Copyright © 1998 Rational Software, all rights reserved

Attributes Help with Impact Assessments

- Requirement attributes provide a link between requirements and other project elements
  - Schedules, design elements, resources, costs, hazards
- For example:
  - Status would provide information about current status of the requirement
  - Effort would allow the developers to rate the estimated work associated with each requirement
- If a change to an existing requirement is suggested
  - Check the status to see if it has been implemented already
  - Check links to lower tier requirements and design elements to determine the impact to progress

Copyright © 1998 Rational Software, all rights reserved

Configuration and Change Management

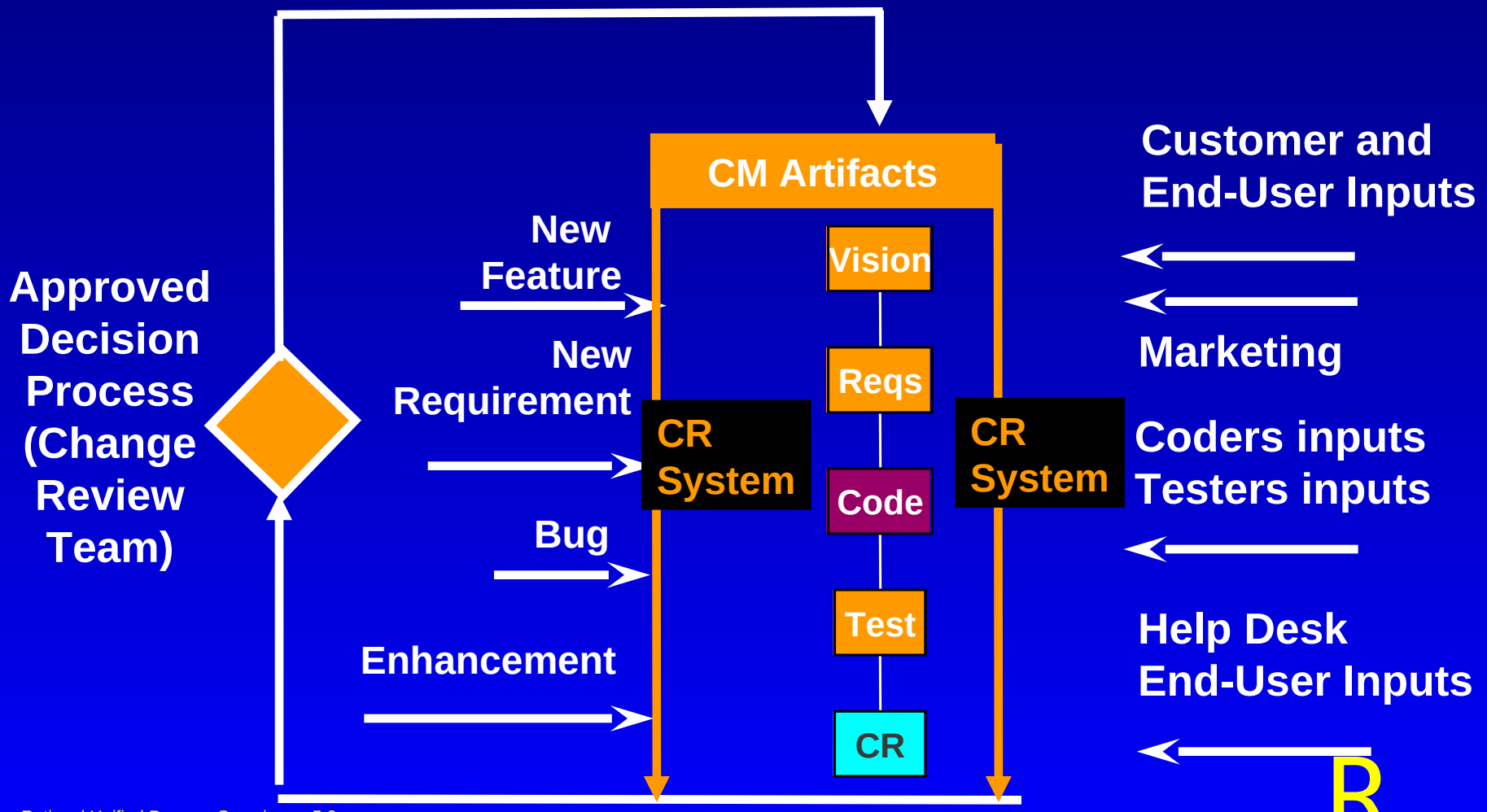
- Configuration and change management of requirements is important for the same reasons that configuration management of source code is important
- Benefits of requirements management under CM
  - Prevent unauthorized change
  - Preserve revisions of requirements documents
  - Retrieve previous versions of documents
  - Allow a managed baseline release strategy
  - Prevent simultaneous update of documents

Copyright © 1998 Rational Software, all rights reserved

R

Change Management Is Key

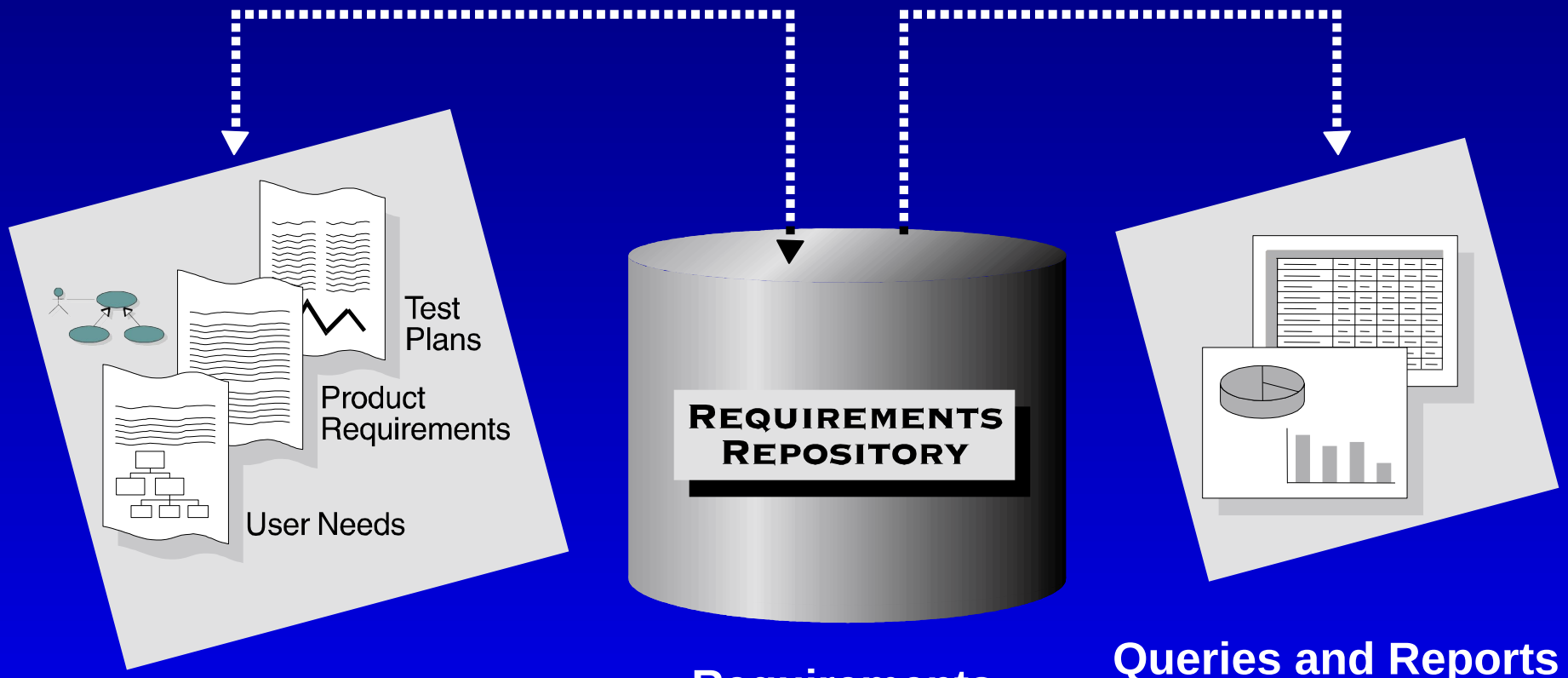
Change requests come from many sources throughout the product lifecycle



Copyright © 1998 Rational Software, all rights reserved

Manage Changing Requirements

Requirements Management Architecture



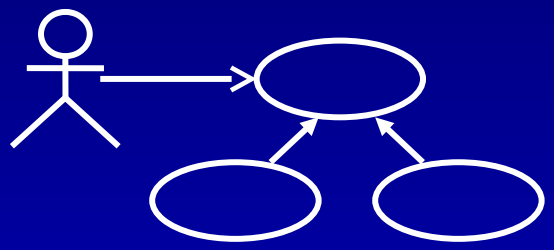
Requirement Documents

Requirements, Attributes, and Traceability

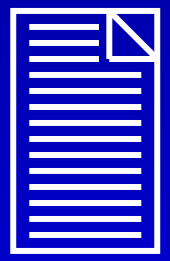
Queries and Reports

Copyright © 1998 Rational Software, all rights reserved

Use Cases Drive Analysis & Design

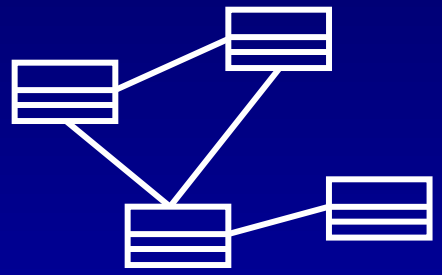


Use-Case Model

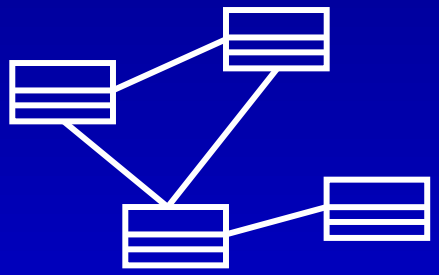


Supplementary Specifications

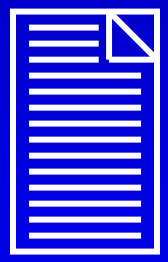
Analysis and Design



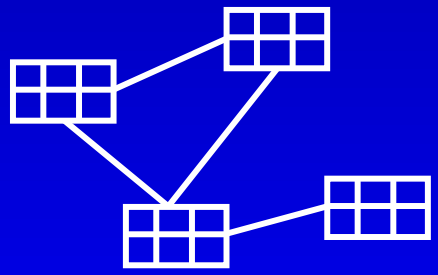
Analysis Model (optional)



Design Model



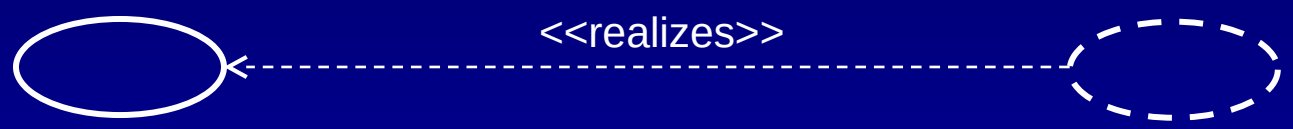
Architecture Document



Data Model

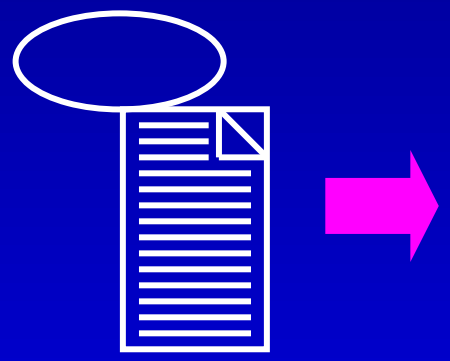
Copyright © 1998 Rational Software, all rights reserved

Use Case Realization in Analysis & Design

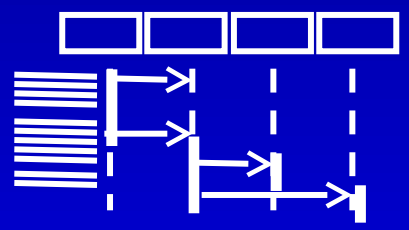


Use Case
(Use-Case Model)

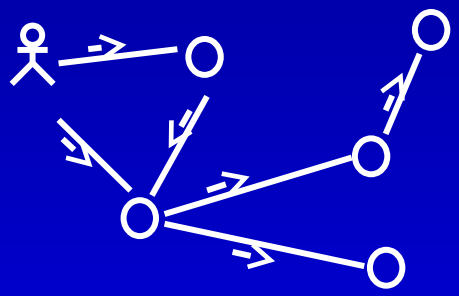
Use-Case Realization
(Design Model)



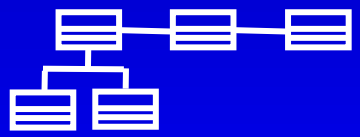
Use Case



Sequence Diagrams



Collaboration Diagrams

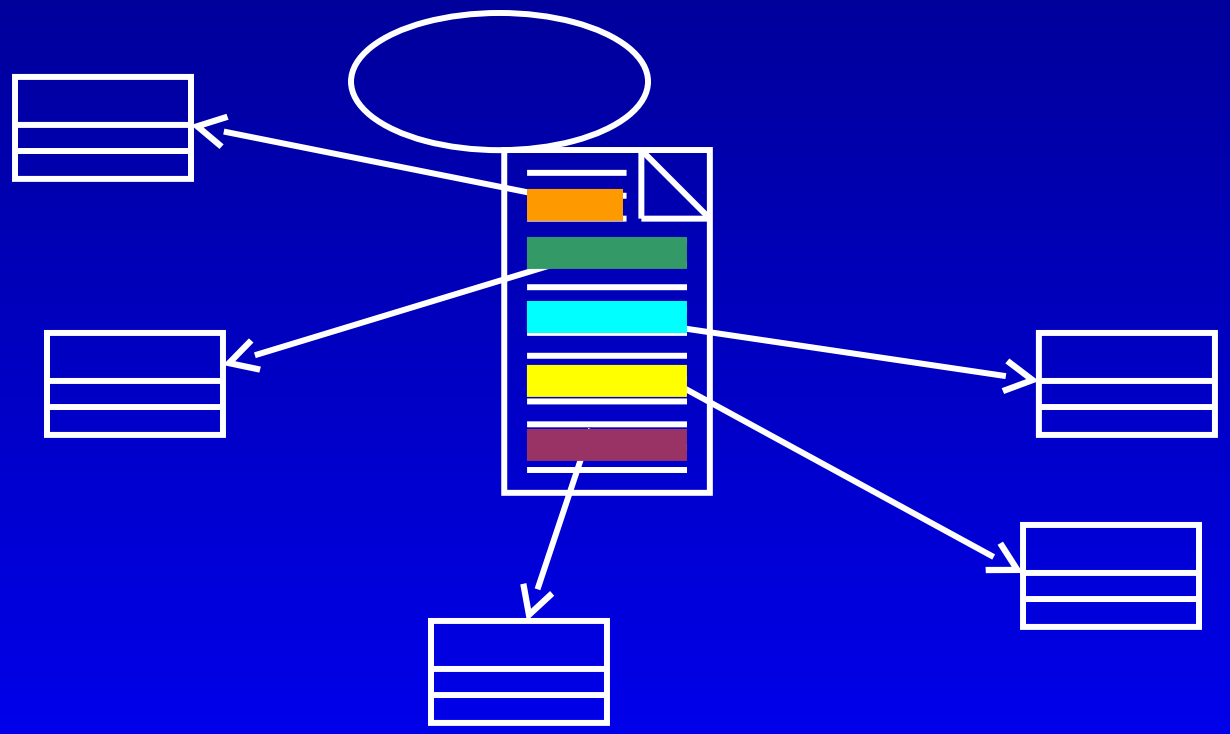


Class Diagrams

Copyright © 1998 Rational Software, all rights reserved

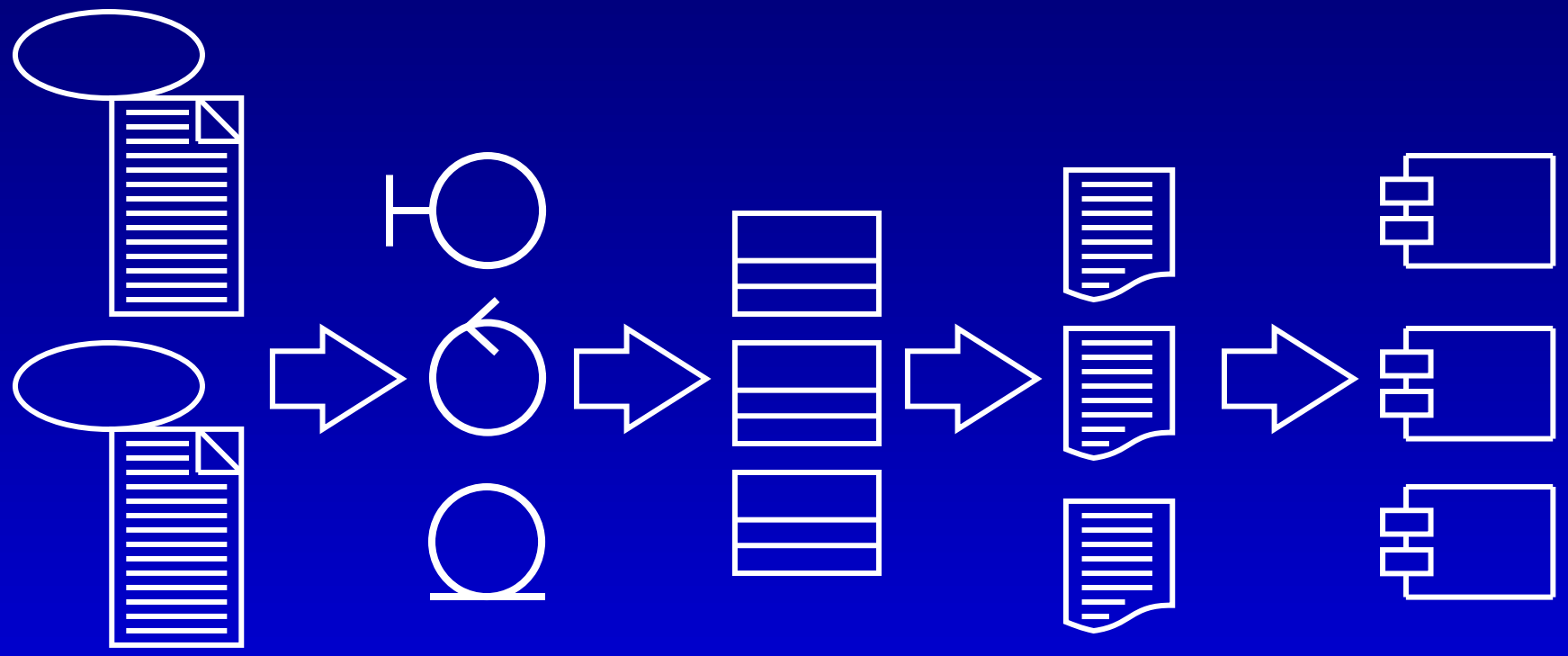
Use-Case Analysis and Design

- The complete behavior of a use case is distributed to collaborating classes



Copyright © 1998 Rational Software, all rights reserved

From Use Cases to Executables

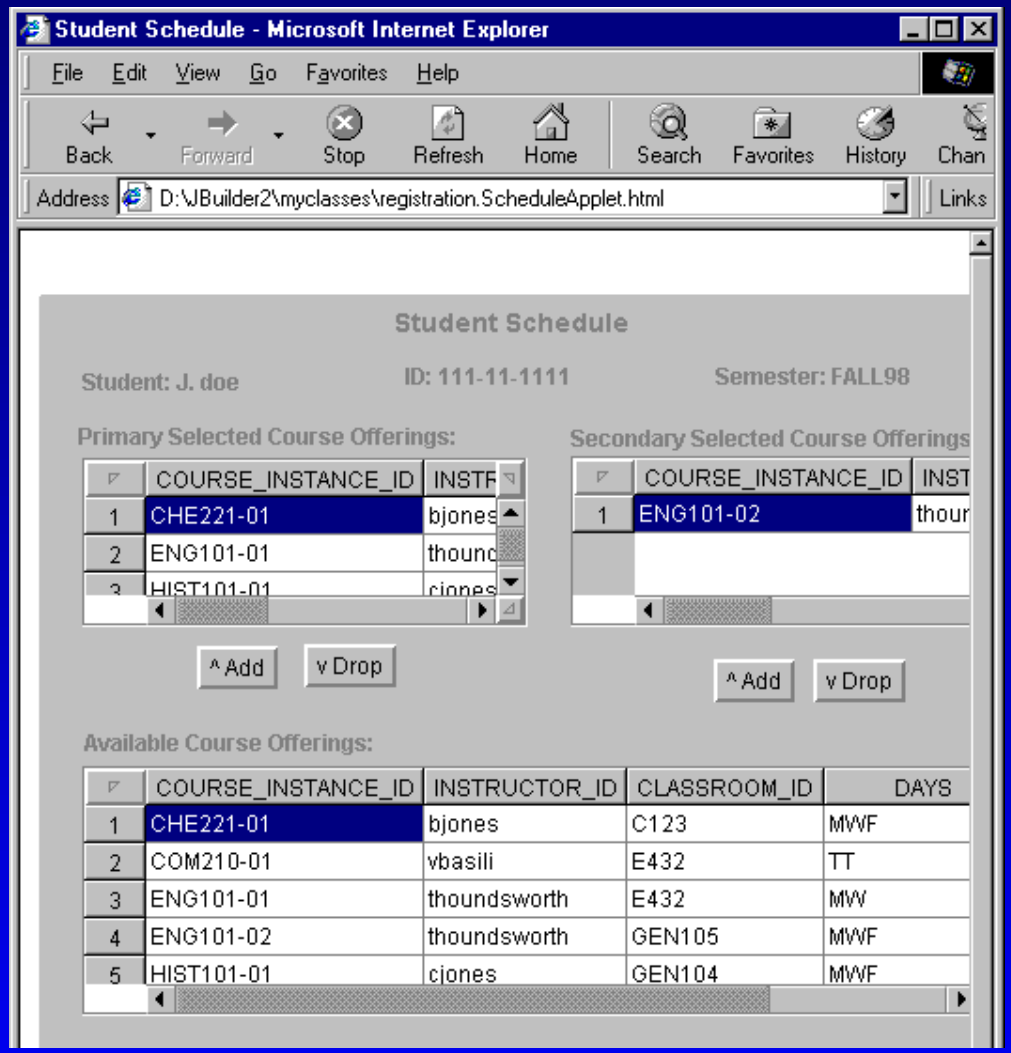


Use Cases | Analysis Classes | Design Classes | Source Code | Exec

Copyright © 1998 Rational Software, all rights reserved

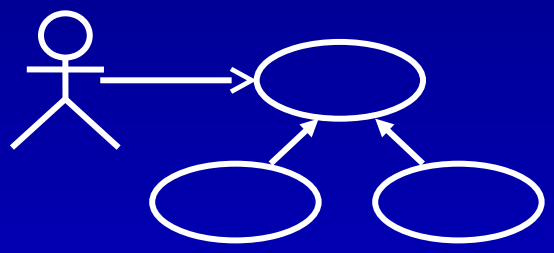
Use Cases Help in Defining User Interfaces

- Sketches of the system's proposed screen appearance can be derived from use cases
- These form the basis of user interface design



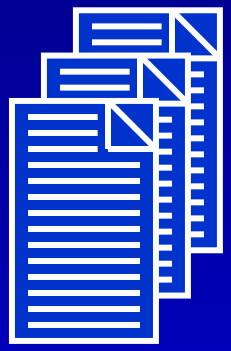
Copyright © 1998 Rational Software, all rights reserved

Use Cases and End-User Documentation



Use-Case Model

Deployment



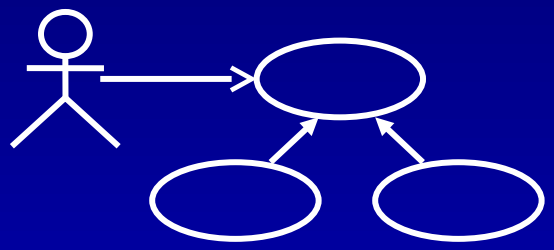
<u>End User Support Material</u>

- User's Guide
- On-line Help
- Demos
- Tutorials
- Training Material

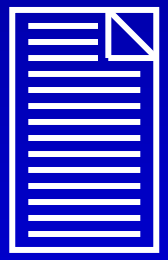
Copyright © 1998 Rational Software, all rights reserved

R

Use Cases Drive the Test Model

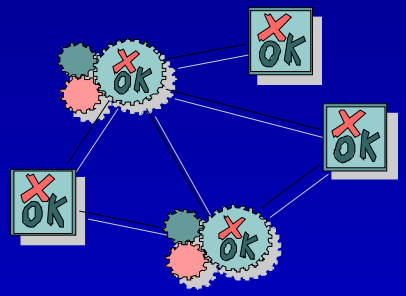


Use-Case Model



Supplementary Specifications

Test



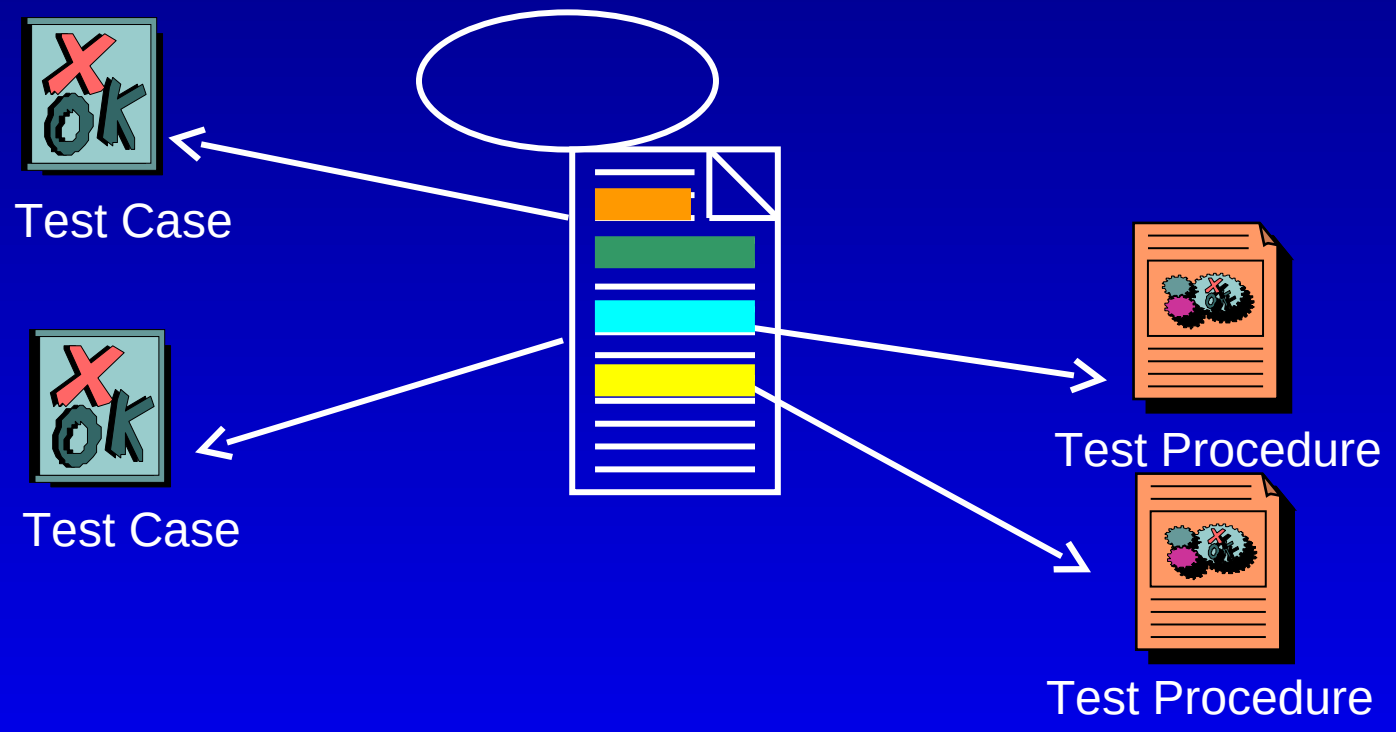
Test Model

Copyright © 1998 Rational Software, all rights reserved

R

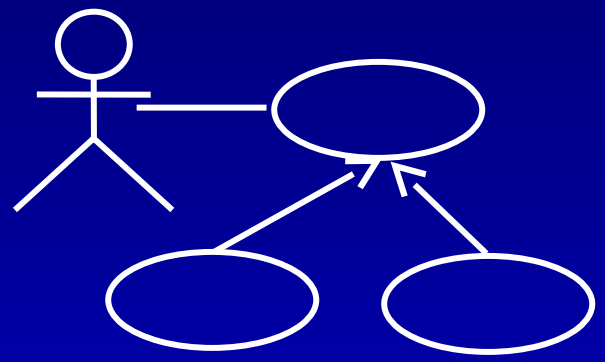
Testing Use Cases

- The complete behavior of a use case is tested using test cases and test procedures



Copyright © 1998 Rational Software, all rights reserved

Use Cases Help in Iteration Planning



Use-Case Model

Supplementary Specification

Constraints

Project Management

Iteration Plan

Fine-grained plan for a single iteration

Copyright © 1998 Rational Software, all rights reserved

R

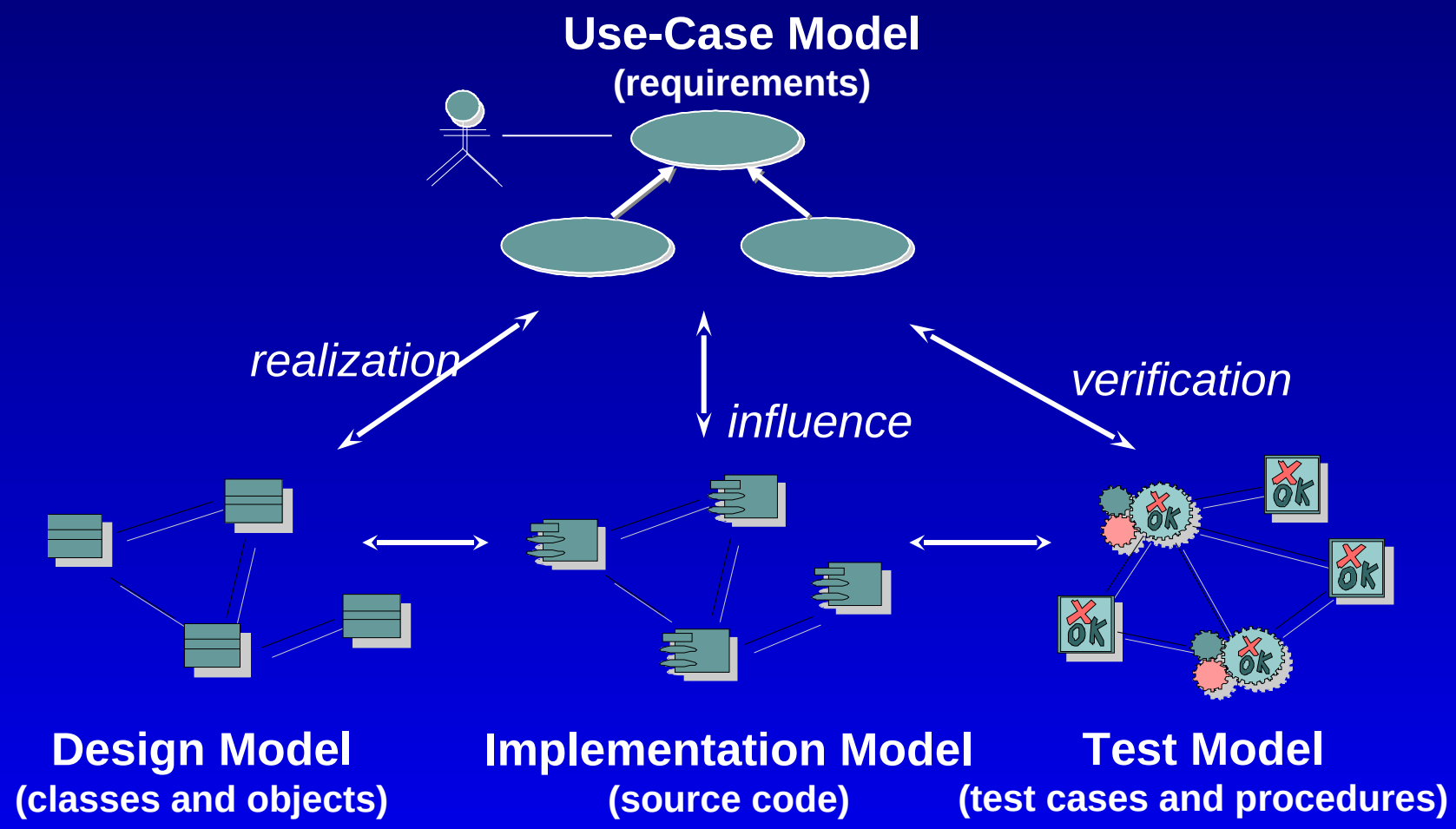
Iteration Planning Activities

- The scope of each iteration is based on
  - The top risks to the project
  - The functionality required of the system
  - The time allocated to the iteration in the project plan
  - The phase and its specific objectives
- Iteration scope is expressed in terms of
  - Selected use cases
  - Selected non-functional requirements

Work to be done in an iteration derives from the iteration scope and hence the use cases

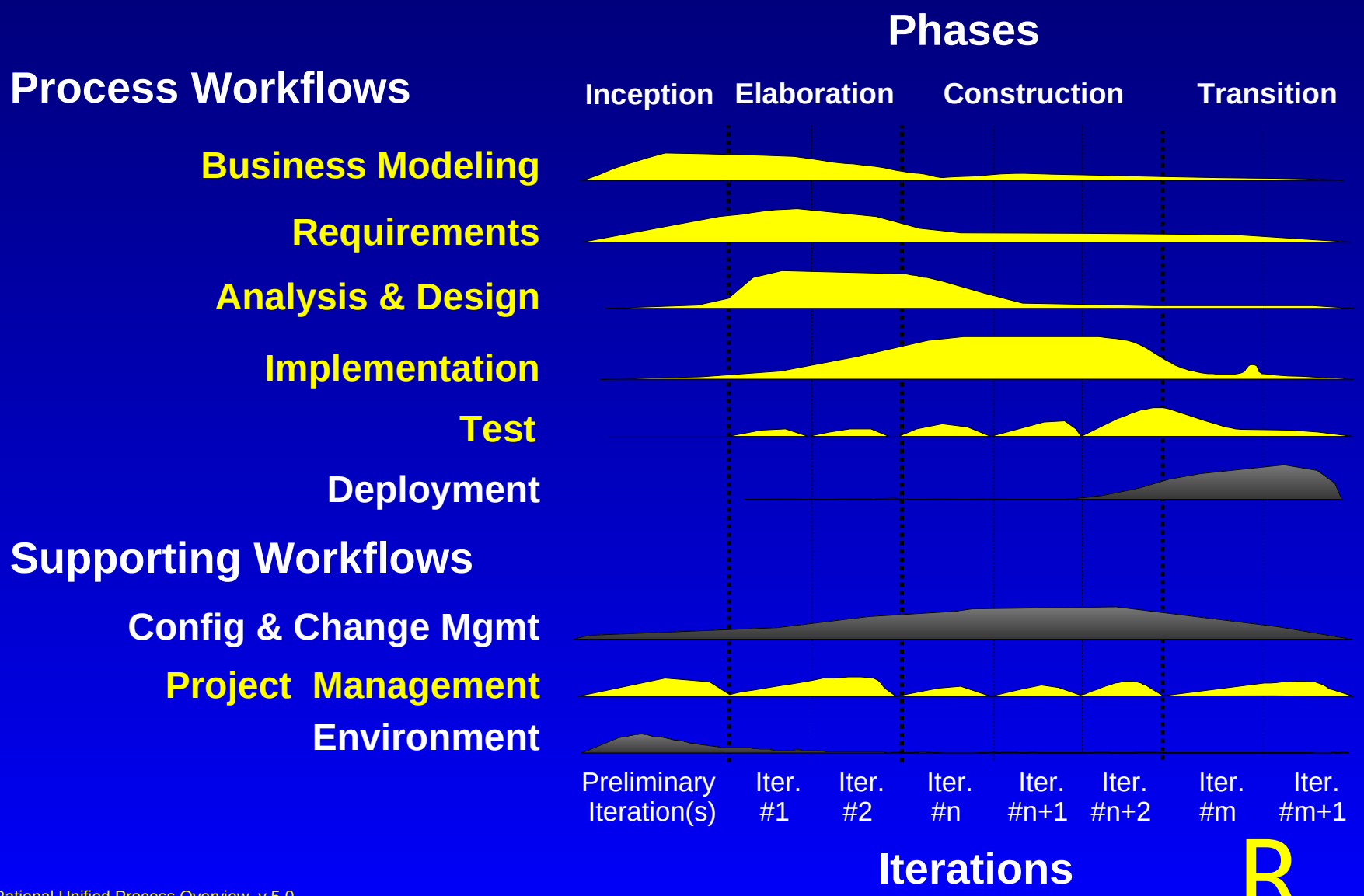
Copyright © 1998 Rational Software, all rights reserved

The Use Case Model Relates to Other Models



Copyright © 1998 Rational Software, all rights reserved

A "Use-Case-Driven" Approach



Copyright © 1998 Rational Software, all rights reserved

Summary: Use-Case-Driven Development

- The use-case model consists of actors, use cases, flow of events, preconditions, postconditions, use-case diagrams and all other information needed to fully define use cases
- Actors interact at the system boundary and therefore help to define system scope
- Attributes can be attached to requirements to record details such as priority and status. Attributes can also link requirements to other requirements and to other project artifacts such as design models
- Use cases are primary inputs to many workflows and drive the activities of analysis & design, test, and project management

Copyright © 1998 Rational Software, all rights reserved